Parallelism

1. A program should be parallel and efficient. FAST!=EFFICIENT. Just because your program runs faster on a parallel computer, it does not mean it is using hardware efficiently. In designing parallel programs, the only things that matter is *communication* and *moving stuff around*.
2. what is a computer program? a list of instructions. The list of commands modifies state in a machine. If you perform these operations on this state, at the end of the day you should get this answer.
3. what does a processor do?
    1. Executes instructions; Modifies state, 2 types of state;
        1. registers
        2. main memory
    2. Very simple processor:
        1. Fetch and Decode (Control); determine what instruction to run next.
        2. ALU: execution unit; performs the opration described by an instruction, which may modify values in the processor's registers or computer's memory
        3. Two forms of context:
            1. Registers: maintain program state: store value of variables used as inputs and outputs to operations
            2. Memory
4. Not all the instructions depend on the previous on on the sequence.
5. superscalar processor:
    1. The processor can decode and execute up to two instructions per clock
    2. Superscalar execution: processor automatically finds independent instructions in an instruction sequence and can execute them in parallel on multiple execution units.
6. Memory:
    1. memory in abstractly is just an array of bytes. Its part of that state of a program.
    2. What are caches?
        1. A cache is a hardware implementation detail that does not impact the output of a program, only its performance.
        2. caches is another form of storage that's part of the implementation of memory. But if the processor says, I need address 0x4, that value maybe stored in cache and we can get the answer much much faster. A cache is like what's right on my desk, compared to usual memory that is like storing an object in back of the room or storage in my garage. Don't think of implementation they're just addresses. I have this concept, I have memory. Memory takes an address it gives a value. DRAM is one way to implement that, a cache is another one to implement that. Why keep DRAM so far? the answer is why put a garage on my desk?
    3. There's to details how a data cache works:
        1. even though memory says, if you give me an address, I'll give you a value, the underlying implementation in any modern system that I know about will move data from memory into the cache at some larger **granularity** of **cache lines**.
        2. One of the big reasons for a cache, for example, If I'm touching stuff a lot, the paper I'm working on right now, the homework I'm grading right now, that's why I'm keeping on my desk, I don't take it to the garage; temporal locality (back-to-back accesses in time). This **granularity** of operating at the granularity of lines, first of all, bulk transfers are almost always efficient, so I can transfer data efficiently to a cache. But it's based on the assumption that if I access address 0, often times programs then go access 1, 2 and 3 too, because they're running right down an array for example.
        3. Note: example of granularity, as an ml intern -> I love this lecture, I process in a granularity of 4 words.
        4. They're two forms of reducing latency here.
            1. Almost like the line is functioning like a prefetch for the next items in the array.
            2. by bringing the data in now, I'm assuming you're probably going to touch it again. A common thing would be to read something, do some logic and then update it, so there's already two memory accesses.
        5. cold miss: touching a data address for the first time.
        6. conflict miss: details not important now.
        7. capacity miss: details not important now (only when designing real caches do this matter)
    4. Summary:
        1. caches exists. They are implementation details. If I took the cache out of the computer, you would get the same results, it just may run slower, that's the most important thing.

    2. Caches exists to reduce the latency of memory access to these processors are not waiting.
    3. just like we store things, in a desktop, a filing cabinet, a hallway closet, and a garage, and I put different stuff in those different storages based on how often I might need them, most modern computers have a hierarchy of various caches. And a reasonable rule of thumb is that if a unit of storage is bigger, it is farther away, and it has higher latency to access. And we'll also talk about how it actually has higher energy to access and stuff like that. If we rip open most modern machines, you usually see a L1 cache, that's pretty small, a few kilobytes right next to the processor that can be accessed in a few cycles. If you miss the L1, you'll ask the L2, if you miss the L2, often you'll see an L3. And if you miss the L3 you'll go out to DRAM.
    4. Data access times (no. of cycles at 4 GHz):
        1. Data in L1 cache: 4 cycles
        2. Data in L2 cache: 12 cycles
        3. Data in L3 cache: 38 cycles
        4. Data in DRAM : 248 cycles (definitely waiting)
    5. You can use 4 cores, however if you use the wrong memory for example the DRAM, you can be off by a factor of 250 slower. Latency: worst case of accessing memory in DRAM is 1/250th of peak performance waiting.
    6. So the scale of these affects that deal with data movement are enormous.
    7. There are no latency benefits in bypassing L3, L2, L1 caches for memory from DRAM in modern system.
7. Superscalar execution is, the core is given one thread, that thread has a sequence of instructions. Because those instructions are in one thread, they are all reading and writing the same registers, there's only one thread one execution context. If there happens to be the ability to find a couple of those instructions that can be run in parallel, this chip may find up to three (Fig. with a copy of three Fetch/Decode, followed by three copy of ALUs, followed by a single Execution context block).
8. A processor has no inherent capability to look at a larger window and a program and say hey look! this for loop can be parallelized for example, since each iteration in a for loop is technically independent of each other (because of the i++). So why not ask the programmer to tell the processor which part could be parallelized.
9. Can define a 'forall' command that says that this outer for loop can be parallelized for example in computing the sin(x) using fourier series the N terms can be divided in N threads (if using N processors) and the details of how the processor is going to parallelize the program is then upto the N processors. We can for example write a program that spawns 2 threads, main thread compute half (N/2) and anoter thread compute another half (N/2). But it will be harder to write a program that uses 16 threads (for computers with 16 cores). In that case, we let the processor decide how to parallelize the program, forall says we don't care how you parallelize it, we know that this for loop can be parallelized.
10. N threads equals N instruction streams.
11.

```
void sinx(int N, int terms, float* x, float* y) {
    for (int i=0; i<N; i++) {
        float value = x[i];
        float numer = x[i] * x[i] * x[i];
        int denom = 6;  // 3!
        int sign = -1;

        for (int j=1; j<=terms; j++) {
            value += sign * numer / denom;
            numer *= x[i] * x[i];
            denom *= (2*j+2) * (2*j+3);
            sign *= -1;
        }

        y[i] = value
    }
}
```

1. Notice that the operations inside the second for loop is the same for all j (same lines of code). This means that we can add more ALUs to vectorize the operation. This is called SIMD short for single instruction multiple data.
2. Now we can combine that with multiple instruction streams. So If take this new type of core and apply it to a 16-core processor, we have a 16-core processor that runs 16 instruction streams, but the instruction in those streams are processing

8 pieces of data at ones. So altogether our chip has 16 cores, 16 instruction streams, 16 times 8, or 128 execution units, which means we can do work on 128 pieces of data at ones, so arguably we could go 128 times faster on the 16-core chip.

3. Summary: we have 16 threads running on a 16-core chip, and those threads, the instruction stream, contains instructions that are processing 8-wide vectors. At any moment in time there are 128 pieces of work getting done at any point in time. So I we perfectly parallelize and we compare our performance to our original starting processor that had one core, and one ALUs per core, we might get128 times faster.
4. Packing a processor with more arithmetic units require only few resources compared to everything else, esp. those that are not high-precision floating point. The main idea is those things are cheap, we can pack a chip full of those if the code is set up to use them.
5. In GPU programming; 'coherent execution' is a jargon/slang that means I have a program where all of the iterations of my loop, everything that's going on, *needs the same instruction*. And the lack of coherence is often called divergence. We can think of divergence as having different control paths are needed, so divergent code will not run well on a SIMD architecture like this. Coherent code will run at high utilization.
6. Implicit SIMD
    1. compiler generates a binary with scalar instructions.
    2. But N instances of the program are always run together on the processor
    3. Hardware (not compiler) is responsible for simultaneously executing the same instruction from multiple program instances on different data on SIMD ALUs.
7. SIMD width of most modern GPUs ranges from 8 (Intel and AMD) to 32.
    1. Divergent execution can be a big issue (poorly written code might execute at 1/32 the peak capability of the machine!)
8. Summary: three different forms of parallel execution (orthogonal ideas)
    1. Superscalar execution
        1. A hardware implementation detail that requires no change of the program at all, which is if there are independent instructions in one instruction stream, the hardware figures it out and can execute two of them in parallel, inside the same core, inside the same instruction stream.
    2. SIMD: the program itself, most of the time says, here are 8 operations or 32 operations that can be done in parallel. I have one instruction stream lot, usually, and it's issuing these vector or larger operations. Note: this statement is not precise for GPUs, but conceptually its the same.
    3. Multi-core: which is taking one of these cores that can run one instruction stream, maybe with vector instruction, maybe with superscalar, and copying them as many times as the resources we have to fit on the chip. Multi-core is just replicating/copying things.
9. Note: An add instruction is usually add this scalar register to this scalar register. An example of SIMD instruction is please add this 8-wide vector to another 8-wide vector element-wise. So a single instruction does eight scalar operations. Like adding an 8-wide tensor to another 8-wide tensor in Numpy or Pytorch using the plus operation, that is a machine instruction.
10. Caches reduce length of stalls (reduce memory access latency). Processors run efficiently when they access data resident in caches. Caches reduce memory access latency when accessing data that they have recently accessed!. Note: Caches also provide high bandwidth data transfer (recall cache lines).
11. Data prefetching reduce stalls (hides latency).
12. Memory access problem: But what if data hasn't been read recently, so does not reside in cache? And the next piece of data to read is not easily predictable?
13. Even with the 8-wide vector operations and superscalar execution, but if we have to wait for memory to be available the processor would still have wasted time waiting.
14. Idea: Hiding stalls with multi-threading. Create registers that have memory of the processes.
15. Simple idea: if you are waiting for something to happen, you can go do something else. For example when doing the laundry, we will seldomly wait for the washing machine to finish its work, or when boiling water we seldomly wait for the water to boil, most of the time we still have other activities to do while waiting for these processes.
16. **Execution context** represents the state of the instruction stream, or in other words, we could say the state of a thread.